



ENSIIE - HPC/BigData
(Programmation Parallèle)

EXAMEN de

Rattrapage

Durée : 3 heures

julien.jaeger@cea.fr

Toutes les réponses doivent être justifiées.
La qualité de la rédaction est prise en compte.

Première partie

Module 1 - Introduction à la programmation parallèle

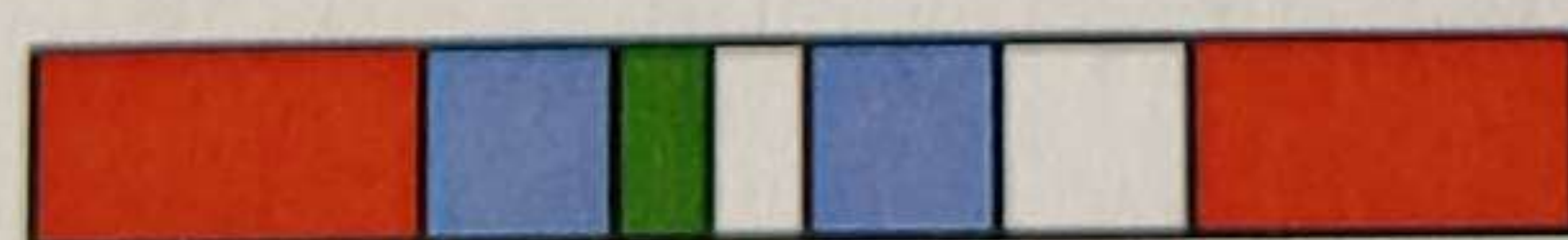
I Questions de cours

Q.1: Quelle est la différence entre programmation parallèle en mémoire partagée et en mémoire distribuée ?

Q.2: Quelles sont les contraintes d'utilisation des communications collectives MPI ?

Q.3: Quel est l'intérêt des appels MPI non-bloquants ? Que veut-on dire par "recouvrement des communications" ?

Q.4: Construire le datatype présenté en figure ci-dessous. Les cases rouges représentent des *double*, les cases bleues des *int* et les cases vertes des *short*. Les cases blanches sont des "trous" dans le nouveau datatype.



II Implémentation d'une réduction/distribution

Soit la fonction :

```
void reduction_distribution_somme( int *sndbuf, int *rcvbuf, int rcvcount );
```

avec

- sndbuf (in) : tableau d'entiers pré-alloué à la taille P rcvcount
- rcvbuf (out) : tableau d'entiers pré-alloué à la taille rcvcount
- rcvcount (in) : nombre d'éléments à recevoir par processus (tous les processus appellent la fonction avec la même valeur de rcvcount)

Cette fonction est une opération collective : appelée par tous les processus MPI, elle effectue :

- une réduction somme sur tous les éléments du vecteur sendbuf,
- puis découpe le résultat en P sous-vecteurs de taille recvcount,
- et distribue le vecteur réduit sur tous les processus : les recvcount premiers éléments du résultat vont au premier processus, les recvcount suivants vont au deuxième processus, etc... jusqu'au derniers recvcount éléments qui vont au dernier processus.

Exemple avec 3 processus MPI		
données P0	données P1	données P2
Entrées : - recvcount = 2 - sendbuf = [1, 3, 4, 0, 6, 5]	Entrées : - recvcount = 2 - sendbuf = [0, 2, 2, 1, 1, 0]	Entrées : - recvcount = 2 - sendbuf = [4, 3, 0, 4, 2, 1]
Sortie : - recvbuf = [5, 8]	Sortie : - recvbuf = [6, 5]	Sortie : - recvbuf = [9, 6]

Q.5: Proposez une implémentation de la fonction *reduction_distribution_somme* en utilisant uniquement des communications collectives avec racine.

III Echanges de mailles fantômes (1)

Pour cette partie, nous allons considérer un code de stencil.

Pour rappel un stencil est un code opérant sur une grille cartésienne. La mise à jour d'une case de cette grille nécessite le concours de case voisine. Dans notre cas, nous allons considérer une grille de 2D. Pour mettre à jour une case, nous avons besoin des 8 cases voisines. En bord de grille, nous considérons que les cases inexistantes apportent une contribution de valeur 0 lors de la mise à jour de la case.

Pour être plus efficaces, nous décidons de faire ces mises à jour en parallèle sur plusieurs processus MPI. Les cases de la grille sont réparties sur H processus en hauteur et L processus en largeur, pour un total de $H \times L$ processus (avec $H > 1$ et $L > 1$).

Q.6: Dans le cas général (pas de conditions aux bords), donner le rang de chaque voisin à partir du rang de la case courante (*mpirank*), de H et de L .

Q.7: Pour chaque direction, quelles sont les conditions à vérifier pour s'assurer de la présence d'un voisin ?

Q.8: A chaque itération, il est nécessaire de récupérer les cases au bord de chaque domaine voisin pour calculer la nouvelle mise à jour. En vous servant des questions précédentes, réaliser un code copiant les données à envoyer dans un buffer temporaire pour chacun des voisins, et réalisant les échanges de mailles fantômes entre les voisins.

Deuxième partie

Module 2 - Programmation parallèle avancées

IV Remote Memory Access (RMA) - One-Sided

Q.9: Adapter le cœur de code d'une implémentation d'un Broadcast en arbre binaire ci-dessous pour utiliser des fonctions one-sided (MPI_Put ou MPI_Get).

```
1  int n = atoi(argv[1]);
2  char *buf;
3  buf = (char*) malloc(n*sizeof(char));
4  if (rank == /*root=*/0) memset(buf, 0, n);
5
6  /* Calcul de la plus petite puissance de 2 >= a nproc */
7  int tmp = nproc >> 1;
8  int p2 = 0;
9
10 while(tmp > 0)
11 {
12     tmp = tmp >> 1;
13     p2++;
14 }
15
16 int log2_nproc = ((1 << p2) == nproc ? p2 : p2+1);
17
18 int tag = 0;
19 int i, two_pi, two_pim1, src, dest;
20
21 for(i = log2_nproc ; i > 0 ; i--)
22 {
23     two_pi = 1 << i;
24     two_pim1 = 1 << i-1;
25
26     dest = -1;
27     if (rank \% two_pi == 0)
28     {
29         dest = rank + two_pim1;
30         /* blindage pour nproc non puissance de 2 */
31         dest = (dest < nproc ? dest : -1);
32     }
33
34     src = -1;
35     /* on verifie que la source obeit au critere d'envoi */
36     if ((rank - two_pim1) >= 0 && (rank - two_pim1) \% two_pi == 0)
37     {
38         src = rank - two_pim1;
39     }
40
41     if (dest >= 0)
42     {
43         MPI_Ssend(buf, n, MPI_BYTE, dest, tag+i, MPI_COMM_WORLD);
44     }
45     else if (src >= 0)
46     {
47         MPI_Recv(buf, n, MPI_BYTE, src, tag+i, MPI_COMM_WORLD,
48                 MPI_STATUS_IGNORE);
49     }
50 }
51 }
```

V Questions de cours

Q.10: Quelles sont les contraintes d'utilisation des collectives non-bloquantes ?

Q.11: Quelles sont les différentes façons de faire progresser des communications asynchrones ? Quels sont leurs avantages et inconvénients ?

Q.12: Quelle est la différence entre un etype et un filetype ? Comment interviennent-ils dans la construction d'une view d'un fichier ?

Q.13: De façon général (hors MPI), quels sont les différents types d'I/O parallèles possibles ? Donnez leurs avantages et leurs inconvénients.

VI Echanges de mailles fantômes (2)

Pour cette partie, nous considérons la même configuration que dans la section III.

Q.14: Donner les fonctions (et leurs paramètres) permettant de construire la topologie cartésienne décrite précédemment.

Q.15: Réaliser les échanges de mailles fantômes dans le stencil avec des collectives neighborhood. Vous pourrez vous inspirer du travail réalisé en Question 8.